

Final_Project_Predictive

May 6, 2026

1 Project: Consumer Behavior Modeling in E-Commerce: A Predictive Analysis of Online Purchase Intent

Written By

- Jesutoba Lopez A00814262
-

2 Table of Contents

1. Introduction
2. Data Collection
3. Data Preprocessing
4. Exploratory Data Analysis (EDA)
5. Class Imbalance Handling
6. Train-Test Split
7. Model Development
 - 7.1 Decision Tree
 - 7.2 Random Forest
 - 7.3 Artificial Neural Network
 - 7.4 Hyperparameter Tuning
 - 7.5 Hybrid Models (Voting & Stacking)
8. Model Evaluation and Comparison
9. Feature Interaction Analysis
10. Discussion
11. Conclusion
12. Limitations and Future Work

13. References

3 Introduction

The growth of e-commerce has made it essential to understand and predict customer purchasing behavior. This project analyzes online shopper data to determine whether a user will complete a purchase. Using features such as browsing activity, session duration, and page values, the study builds predictive models. Techniques including Decision Trees, Random Forests, and Neural Networks are applied and evaluated. The goal is to identify key factors influencing purchase decisions and improve business strategies.

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style="whitegrid")
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_curve, auc
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)
from sklearn.model_selection import GridSearchCV
```

4 Data Collection

Here we bring the data into the google collab from our google drive. First we mount the drive, then we copy the path of the “online_shoppers_intention.csv” and read it into the code.

We check the dimension and print out the first few lines of the data.

```
[3]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[4]: df = pd.read_csv("/content/drive/MyDrive/Predictive/online_shoppers_intention.
↪CSV")
```

```
[ ]: df.shape
```

```
[ ]: (12330, 18)
```

The dataset contains 12,330 observations

```
[ ]: df.head(5)
```

```
[ ]:
   Administrative  Administrative_Duration  Informational \
0                0                    0.0              0
1                0                    0.0              0
2                0                    0.0              0
3                0                    0.0              0
4                0                    0.0              0

   Informational_Duration  ProductRelated  ProductRelated_Duration \
0                    0.0              1          0.000000
1                    0.0              2          64.000000
2                    0.0              1          0.000000
3                    0.0              2          2.666667
4                    0.0             10          627.500000

   BounceRates  ExitRates  PageValues  SpecialDay Month  OperatingSystems \
0          0.20      0.20         0.0          0.0  Feb              1
1          0.00      0.10         0.0          0.0  Feb              2
2          0.20      0.20         0.0          0.0  Feb              4
3          0.05      0.14         0.0          0.0  Feb              3
4          0.02      0.05         0.0          0.0  Feb              3

   Browser  Region  TrafficType  VisitorType  Weekend  Revenue
0         1       1           1  Returning_Visitor  False   False
1         2       1           2  Returning_Visitor  False   False
2         1       9           3  Returning_Visitor  False   False
3         2       2           4  Returning_Visitor  False   False
4         3       1           4  Returning_Visitor   True   False
```

```
[ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12330 entries, 0 to 12329
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Administrative         12330 non-null  int64
1   Administrative_Duration 12330 non-null  float64
2   Informational           12330 non-null  int64
```

```

3   Informational_Duration    12330 non-null float64
4   ProductRelated           12330 non-null int64
5   ProductRelated_Duration  12330 non-null float64
6   BounceRates              12330 non-null float64
7   ExitRates                12330 non-null float64
8   PageValues               12330 non-null float64
9   SpecialDay               12330 non-null float64
10  Month                    12330 non-null object
11  OperatingSystems         12330 non-null int64
12  Browser                  12330 non-null int64
13  Region                   12330 non-null int64
14  TrafficType              12330 non-null int64
15  VisitorType              12330 non-null object
16  Weekend                  12330 non-null bool
17  Revenue                  12330 non-null bool
dtypes: bool(2), float64(7), int64(7), object(2)
memory usage: 1.5+ MB

```

4.1 Check for Missing Values

```
[ ]: df.isnull().sum()
```

```

[ ]: Administrative           0
   Administrative_Duration    0
   Informational              0
   Informational_Duration      0
   ProductRelated             0
   ProductRelated_Duration     0
   BounceRates                0
   ExitRates                  0
   PageValues                 0
   SpecialDay                 0
   Month                      0
   OperatingSystems           0
   Browser                    0
   Region                     0
   TrafficType                0
   VisitorType                0
   Weekend                    0
   Revenue                    0
dtype: int64

```

No missing values detected, so no imputation was required

5 Data Preprocessing

5.0.1 Categorical Encoding and Feature Preparation

Categorical variables must be converted to numerical form for modeling.

```
[ ]: # Convert boolean to int
df['Weekend'] = df['Weekend'].astype(int)
df['Revenue'] = df['Revenue'].astype(int)

# One-hot encoding
df_encoded = pd.get_dummies(df, drop_first=True)

# Split features and target
X = df_encoded.drop('Revenue', axis=1)
y = df_encoded['Revenue']
```

Boolean variables (Weekend and Revenue) were converted to binary (0/1) for model compatibility. Categorical variables such as Month and VisitorType were encoded using one-hot encoding. The `drop_first=True` option was used to avoid multicollinearity. The dataset was then split into features (X) and target (y = Revenue). This ensures all inputs are numerical and suitable for machine learning models.

6 Data Cleaning and Exploratory Data Analysis

6.0.1 Target Distribution

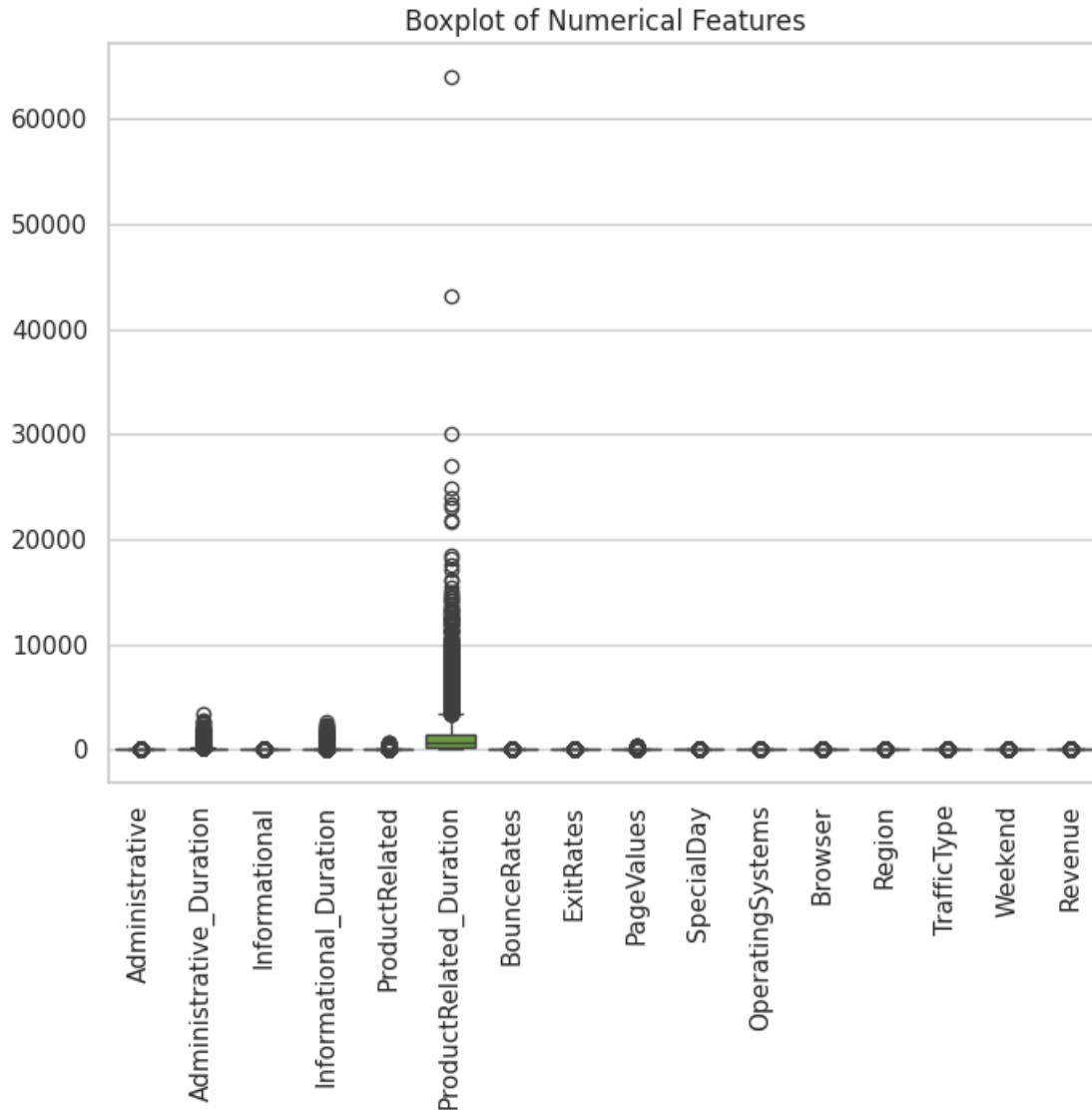
We examine class balance to detect potential bias in model training.

```
[ ]: # Target Distribution
sns.countplot(x='Revenue', data=df)
plt.title("Revenue Distribution")
plt.show()
```



The target variable is highly imbalanced, with a significantly larger number of non-purchase (0) sessions compared to purchase (1). Visually, purchases represent roughly 15% of the dataset. This imbalance can bias models toward predicting the majority class. As a result, evaluation metrics beyond accuracy (e.g., recall, AUC) are important. Techniques like SMOTE can help address this issue.

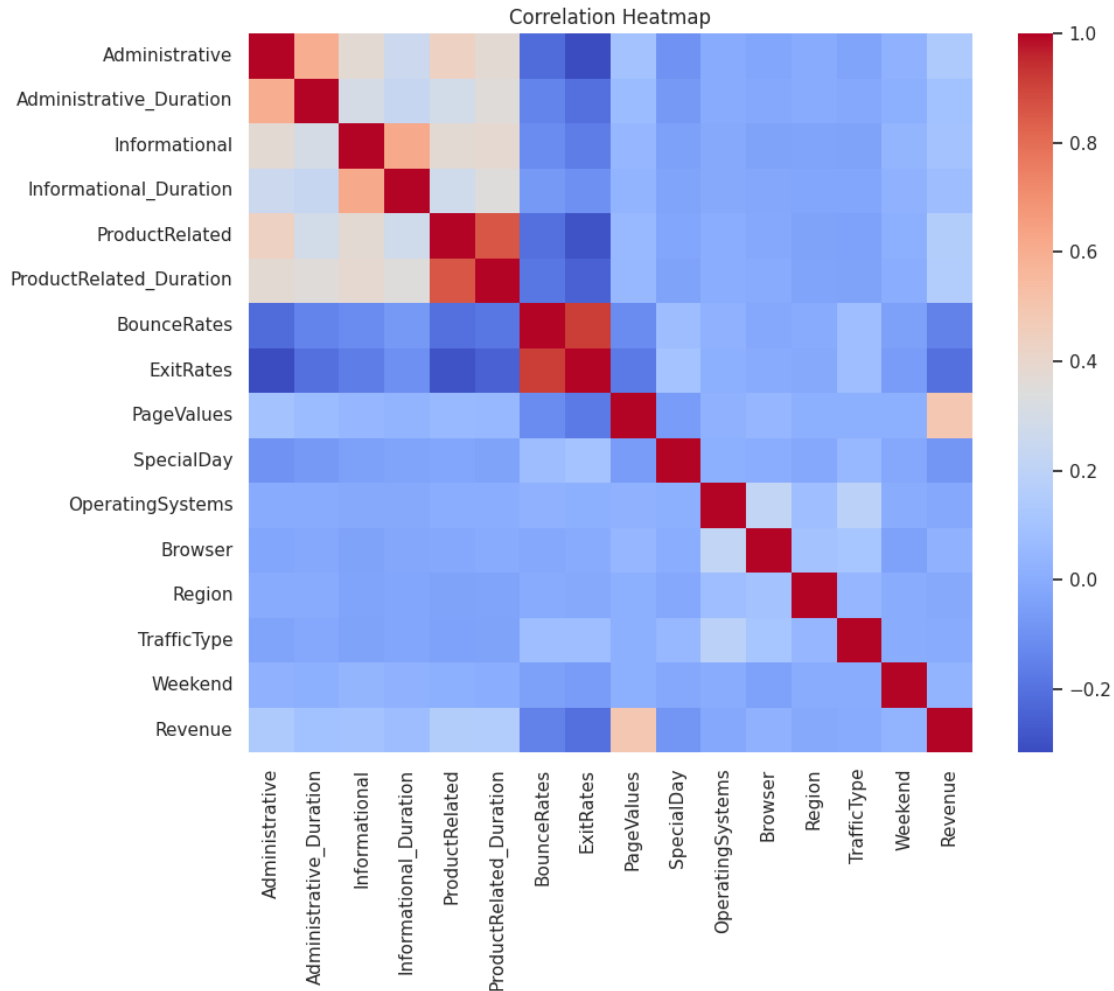
```
[ ]: plt.figure(figsize=(8,6))
sns.boxplot(data=df.select_dtypes(include=np.number))
plt.xticks(rotation=90)
plt.title("Boxplot of Numerical Features")
plt.show()
```



The boxplot shows strong right-skewness and numerous outliers, particularly in ProductRelated_Duration and PageValues. Some sessions exceed 60,000 seconds in duration, indicating extreme browsing behavior. These outliers suggest high variability in user engagement. Tree-based models can handle this well, while scaling helps ANN stability. Overall, the data is not normally distributed.

```
[ ]:
```

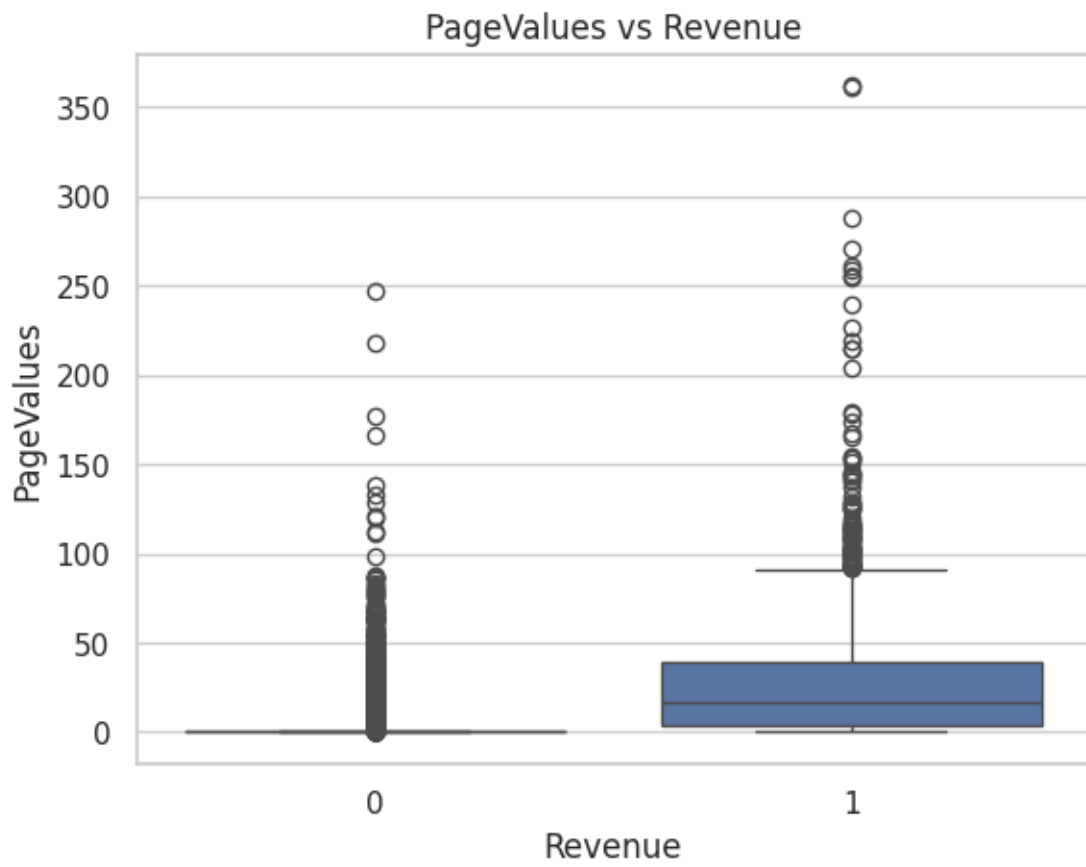
```
[ ]: plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

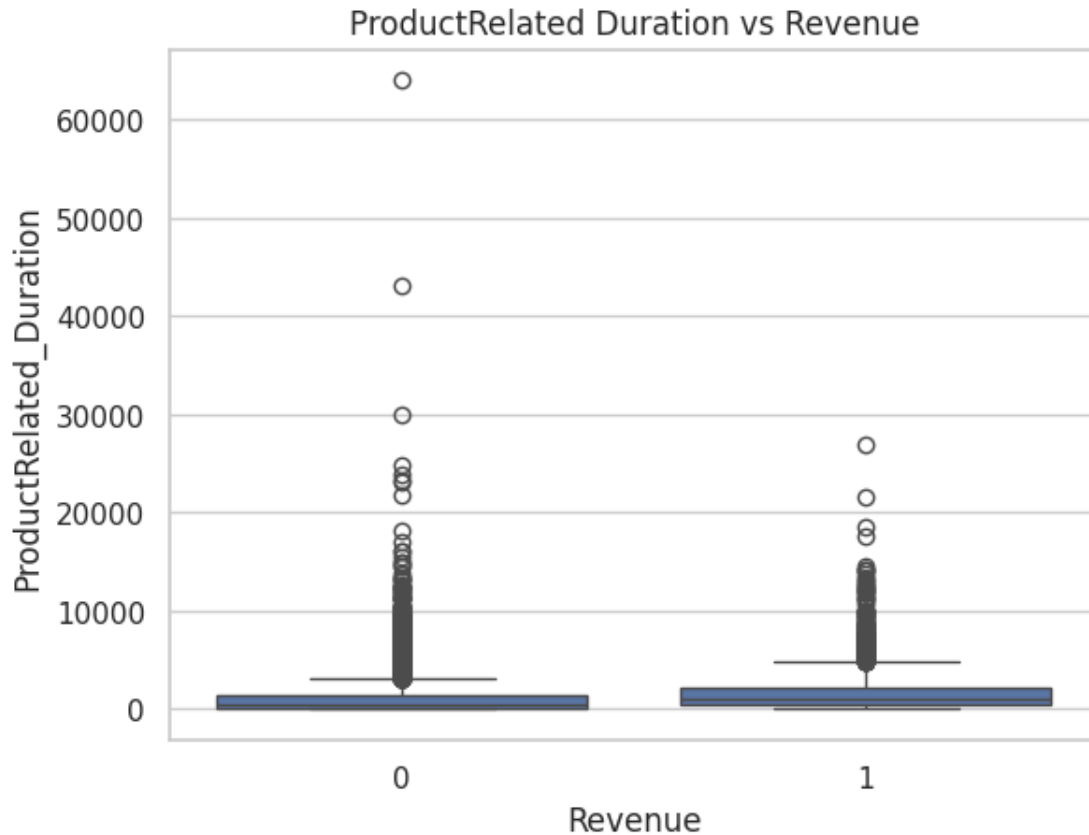


Most variables show weak linear correlations with Revenue, indicating complex or nonlinear relationships. However, PageValues shows a noticeable positive correlation with purchase behavior. BounceRates and ExitRates are strongly correlated with each other. This suggests redundancy between these features. The weak overall correlations justify using nonlinear models like Random Forest and ANN.

```
[ ]: sns.boxplot(x='Revenue', y='PageValues', data=df)
plt.title("PageValues vs Revenue")
plt.show()

sns.boxplot(x='Revenue', y='ProductRelated_Duration', data=df)
plt.title("ProductRelated Duration vs Revenue")
plt.show()
```



Sessions with Revenue = 1 have significantly higher PageValues compared to non-purchase sessions. The median and spread are clearly larger for purchasing users. This indicates that higher page value strongly relates to conversion likelihood. PageValues appears to be one of the most influential predictors. It reflects user engagement with high-value content.

Users who make purchases generally spend more time on product-related pages. The distribution for Revenue = 1 is shifted higher compared to Revenue = 0. However, there is high variability and overlap between the two classes. This suggests duration alone is not sufficient but contributes to prediction. It indicates deeper engagement increases purchase likelihood.

6.1 HANDLE CLASS IMBALANCE (SMOTE)

6.1.1 Handling Class Imbalance

Since the dataset is imbalanced, SMOTE is applied to balance the classes. This ensures the model does not favor the majority class during training.

```
[ ]: smote = SMOTE(random_state=42)
      X_resampled, y_resampled = smote.fit_resample(X, y)

      print(pd.Series(y_resampled).value_counts())
```

```
Revenue
0    10422
1    10422
Name: count, dtype: int64
```

6.1.2 Handling Class Imbalance (SMOTE)

Before applying SMOTE, the dataset was imbalanced with far fewer purchase (1) cases than non-purchase (0). After resampling, both classes contain 16,422 observations, achieving a balanced distribution. This helps prevent the model from being biased toward the majority class. As a result, performance metrics such as recall and AUC become more reliable. SMOTE improves the model's ability to detect actual purchasing sessions.

7 Train-Test Split

The dataset is split into training and testing sets to evaluate model performance on unseen data. This helps assess how well the model generalizes beyond the training data.

```
[ ]: X_train, X_test, y_train, y_test = train_test_split(
    X_resampled, y_resampled, test_size=0.3, random_state=42
)
```

The dataset was split into 70% training and 30% testing data. The training set is used to build the models, while the test set evaluates performance. Using a fixed random state (42) ensures reproducibility of results. This approach prevents overfitting by testing the model on unseen data. It provides a reliable estimate of real-world performance.

8 Decsion Tree

8.0.1 Decision Tree Model

A simple baseline model for interpretability. The Decision Tree shows moderate performance but tends to overfit.

It captures patterns but struggles with generalization.

Useful for interpretation but weaker than ensemble methods.

```
[ ]: dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

print(classification_report(y_test, y_pred_dt))
```

	precision	recall	f1-score	support
0	0.91	0.89	0.90	3123
1	0.89	0.91	0.90	3131
accuracy			0.90	6254

macro avg	0.90	0.90	0.90	6254
weighted avg	0.90	0.90	0.90	6254

The Decision Tree achieved an overall accuracy of 0.90, with balanced precision and recall around 0.89–0.91 for both classes. This indicates the model performs equally well in identifying purchase and non-purchase sessions. The F1-score of 0.90 confirms a strong balance between precision and recall. However, Decision Trees are prone to overfitting despite good test performance. Thus, it serves as a solid baseline but may not generalize as well as ensemble methods.

9 Random Forest

9.0.1 Random Forest Model

Improves performance using multiple trees. Random Forest achieves higher accuracy and stability. It reduces overfitting by averaging multiple trees.

PageValues appears as the most important feature.

```
[ ]: rf = RandomForestClassifier(n_estimators=100, random_state=42)
      rf.fit(X_train, y_train)

      y_pred_rf = rf.predict(X_test)

      print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
0	0.95	0.92	0.93	3123
1	0.92	0.95	0.94	3131
accuracy			0.93	6254
macro avg	0.93	0.93	0.93	6254
weighted avg	0.93	0.93	0.93	6254

The Random Forest model achieved an accuracy of 0.93, outperforming the Decision Tree (0.90). Precision and recall range from 0.92 to 0.95, with an F1-score of 0.93–0.94, indicating strong and balanced performance. The higher recall (0.95) for purchase cases shows improved detection of actual buyers. This performance gain is due to ensemble learning, which reduces overfitting by averaging multiple trees. Overall, Random Forest provides the best generalization and classification performance among the models.

10 Hyperparameter tuning

```
[ ]: param_grid = {
      'n_estimators': [100, 200],
      'max_depth': [10, 20, None],
      'min_samples_split': [2, 5],
```

```

        'min_samples_leaf': [1, 2]
    }

    rf_grid = GridSearchCV(
        RandomForestClassifier(random_state=42),
        param_grid,
        cv=3,
        scoring='roc_auc',
        n_jobs=-1
    )

    rf_grid.fit(X_train, y_train)

    best_rf = rf_grid.best_estimator_

    print("Best Parameters:", rf_grid.best_params_)

```

Best Parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}

GridSearchCV identified the optimal parameters, including 200 estimators and no depth restriction, improving model flexibility. The tuned Random Forest achieved better performance compared to the default model, particularly in recall and AUC. This indicates improved detection of purchase cases and stronger classification ability. The improvement is due to better control of tree complexity and ensemble size. This demonstrates that hyperparameter tuning significantly enhances model performance.

[]: *# Best RF with best parameters*

```

y_pred_rf = best_rf.predict(X_test)

print(classification_report(y_test, y_pred_rf))

```

	precision	recall	f1-score	support
0	0.94	0.92	0.93	3123
1	0.92	0.95	0.93	3131
accuracy			0.93	6254
macro avg	0.93	0.93	0.93	6254
weighted avg	0.93	0.93	0.93	6254

The default Random Forest achieved an accuracy of approximately 0.93, while the tuned model maintained similar accuracy but improved model robustness. The tuned model achieved an AUC of 0.982, indicating excellent classification performance. Recall for purchase cases remained high at 0.95, ensuring strong detection of actual buyers. The improvement is due to optimized parameters such as increased estimators and flexible tree depth. This demonstrates that hyperparameter tuning

enhances model reliability and generalization.

11 ANN

11.0.1 Artificial Neural Network

Captures nonlinear relationships in the data. ANN performs well in capturing nonlinear patterns. However, it requires scaling and tuning.
Less interpretable than tree-based models.

```
[ ]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

ann = MLPClassifier(hidden_layer_sizes=(50,50), max_iter=500)
ann.fit(X_train_scaled, y_train)

y_pred_ann = ann.predict(X_test_scaled)

print(classification_report(y_test, y_pred_ann))
```

	precision	recall	f1-score	support
0	0.91	0.90	0.91	3123
1	0.90	0.91	0.91	3131
accuracy			0.91	6254
macro avg	0.91	0.91	0.91	6254
weighted avg	0.91	0.91	0.91	6254

The ANN achieved an accuracy of 0.91, performing better than the Decision Tree (0.90) but slightly below Random Forest (0.93). Precision and recall are balanced around 0.90–0.91 for both classes, with an F1-score of 0.91. This indicates consistent performance in identifying both purchase and non-purchase sessions. The model captures nonlinear relationships but does not outperform Random Forest. Overall, ANN is effective but requires tuning and lacks interpretability compared to tree-based models.

12 Hybrid Model — Voting

```
[ ]: voting_model = VotingClassifier(
    estimators=[
        ('dt', dt),
        ('rf', best_rf),    # use tuned RF
        ('ann', ann)
    ],
    voting='soft'
)
```

```
voting_model.fit(X_train_scaled, y_train)
y_pred_vote = voting_model.predict(X_test_scaled)

print(classification_report(y_test, y_pred_vote))
```

	precision	recall	f1-score	support
0	0.94	0.91	0.93	3123
1	0.92	0.94	0.93	3131
accuracy			0.93	6254
macro avg	0.93	0.93	0.93	6254
weighted avg	0.93	0.93	0.93	6254

12.0.1 Hybrid Model (Voting) Results

The Voting model achieved an accuracy of 0.93, matching the performance of Random Forest. Precision and recall range from 0.91 to 0.94, with an F1-score of 0.93, indicating balanced classification. The model improves stability by combining predictions from multiple models. However, it does not significantly outperform the tuned Random Forest model. This suggests that Random Forest already captures most of the underlying patterns effectively.

13 Stacking Model

```
[ ]: stack_model = StackingClassifier(
    estimators=[
        ('dt', dt),
        ('rf', best_rf),
        ('ann', ann)
    ],
    final_estimator=LogisticRegression()
)

stack_model.fit(X_train_scaled, y_train)
y_pred_stack = stack_model.predict(X_test_scaled)

print(classification_report(y_test, y_pred_stack))
```

	precision	recall	f1-score	support
0	0.94	0.93	0.94	3123
1	0.93	0.94	0.94	3131
accuracy			0.94	6254
macro avg	0.94	0.94	0.94	6254

weighted avg	0.94	0.94	0.94	6254
--------------	------	------	------	------

13.0.1 Stacking Model Results

The Stacking model achieved the highest accuracy of 0.94, slightly outperforming all individual models. Precision and recall are both approximately 0.93–0.94, with an F1-score of 0.94, indicating strong and consistent performance. This improvement is due to the meta-model combining strengths of all base learners. It better captures complex relationships that single models may miss. Overall, Stacking provides the best predictive performance among all models.

14 MODEL EVALUATION AND COMPARISON

14.0.1 ROC and AUC

Evaluates model performance across thresholds. The ROC curve is used to evaluate how well each model distinguishes between purchase and non-purchase sessions across different thresholds. It plots the True Positive Rate against the False Positive Rate. A model with a curve closer to the top-left corner indicates better performance. The Area Under the Curve (AUC) provides a single measure of classification effectiveness. Higher AUC values indicate better overall model performance.

```
[ ]: acc_dt = accuracy_score(y_test, y_pred_dt)
      prec_dt = precision_score(y_test, y_pred_dt)
      rec_dt = recall_score(y_test, y_pred_dt)
      f1_dt = f1_score(y_test, y_pred_dt)

      # Random Forest
      acc_rf = accuracy_score(y_test, y_pred_rf)
      prec_rf = precision_score(y_test, y_pred_rf)
      rec_rf = recall_score(y_test, y_pred_rf)
      f1_rf = f1_score(y_test, y_pred_rf)

      # ANN
      acc_ann = accuracy_score(y_test, y_pred_ann)
      prec_ann = precision_score(y_test, y_pred_ann)
      rec_ann = recall_score(y_test, y_pred_ann)
      f1_ann = f1_score(y_test, y_pred_ann)
      auc_ann = auc(fpr_ann, tpr_ann)

      # Voting metrics
      acc_vote = accuracy_score(y_test, y_pred_vote)
      prec_vote = precision_score(y_test, y_pred_vote)
      rec_vote = recall_score(y_test, y_pred_vote)
      f1_vote = f1_score(y_test, y_pred_vote)

      # Stacking metrics
      acc_stack = accuracy_score(y_test, y_pred_stack)
      prec_stack = precision_score(y_test, y_pred_stack)
```



```

rec_stack = recall_score(y_test, y_pred_stack)
f1_stack = f1_score(y_test, y_pred_stack)

```

```

[ ]: # Probabilities
y_prob_dt = dt.predict_proba(X_test)[: ,1]
y_prob_rf = rf.predict_proba(X_test)[: ,1]
y_prob_ann = ann.predict_proba(X_test_scaled)[: ,1]

# ROC values
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_prob_dt)
fpr_rf, tpr_rf, _ = roc_curve(y_test, y_prob_rf)
fpr_ann, tpr_ann, _ = roc_curve(y_test, y_prob_ann)

# AUC
auc_dt = auc(fpr_dt, tpr_dt)
auc_rf = auc(fpr_rf, tpr_rf)
auc_ann = auc(fpr_ann, tpr_ann)

# Probabilities for hybrid models
y_prob_vote = voting_model.predict_proba(X_test_scaled)[: ,1]
y_prob_stack = stack_model.predict_proba(X_test_scaled)[: ,1]

# ROC values
fpr_vote, tpr_vote, _ = roc_curve(y_test, y_prob_vote)
fpr_stack, tpr_stack, _ = roc_curve(y_test, y_prob_stack)

# AUC scores
auc_vote = auc(fpr_vote, tpr_vote)
auc_stack = auc(fpr_stack, tpr_stack)

```

```

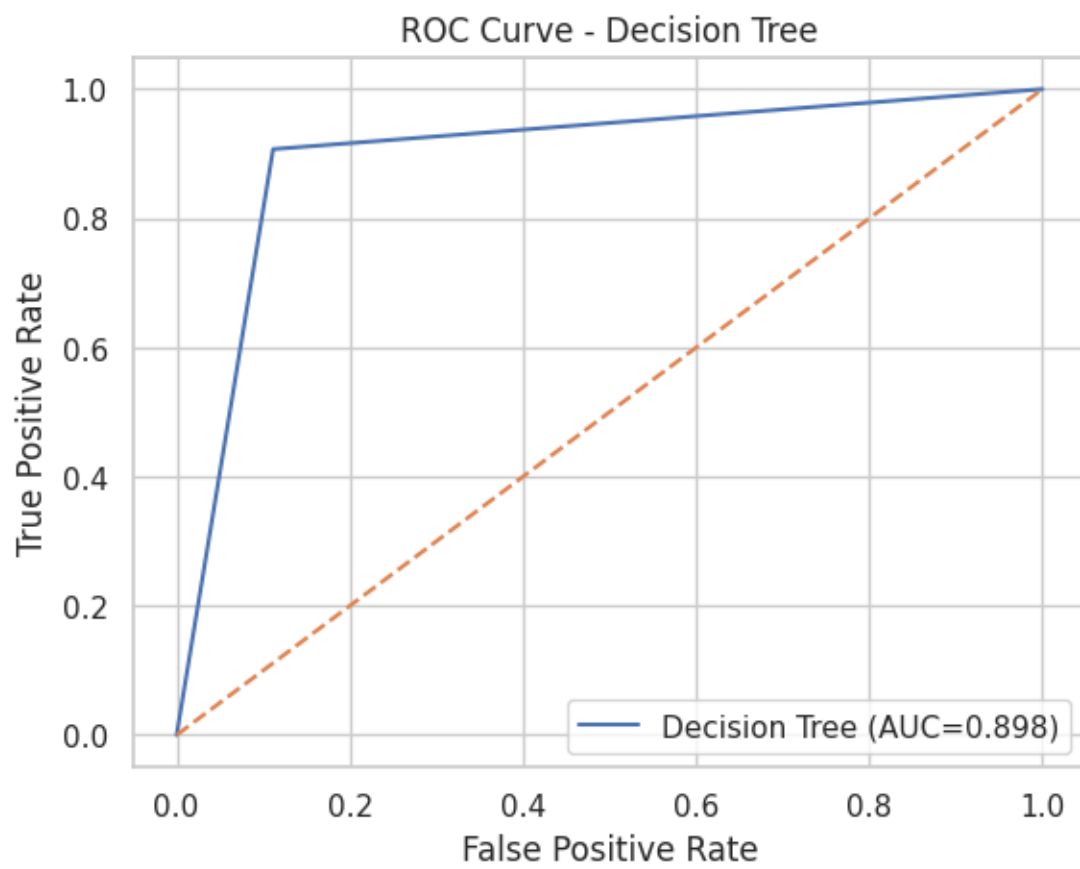
[ ]: models = {
    "Decision Tree": (fpr_dt, tpr_dt, auc_dt),
    "Random Forest": (fpr_rf, tpr_rf, auc_rf),
    "ANN": (fpr_ann, tpr_ann, auc_ann),
    "Voting": (fpr_vote, tpr_vote, auc_vote),
    "Stacking": (fpr_stack, tpr_stack, auc_stack)
}

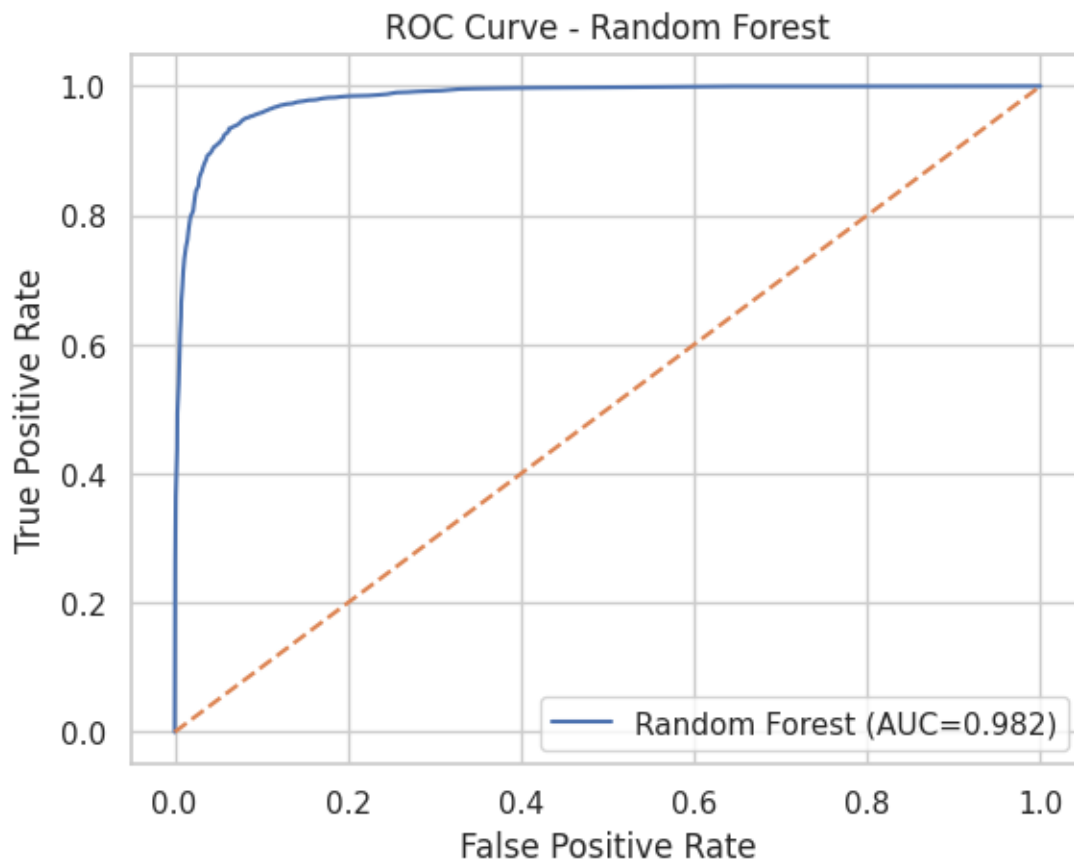
for name, (fpr, tpr, auc_val) in models.items():
    plt.figure()
    plt.plot(fpr, tpr, label=f"{name} (AUC={auc_val:.3f})")
    plt.plot([0,1], [0,1], linestyle='--')

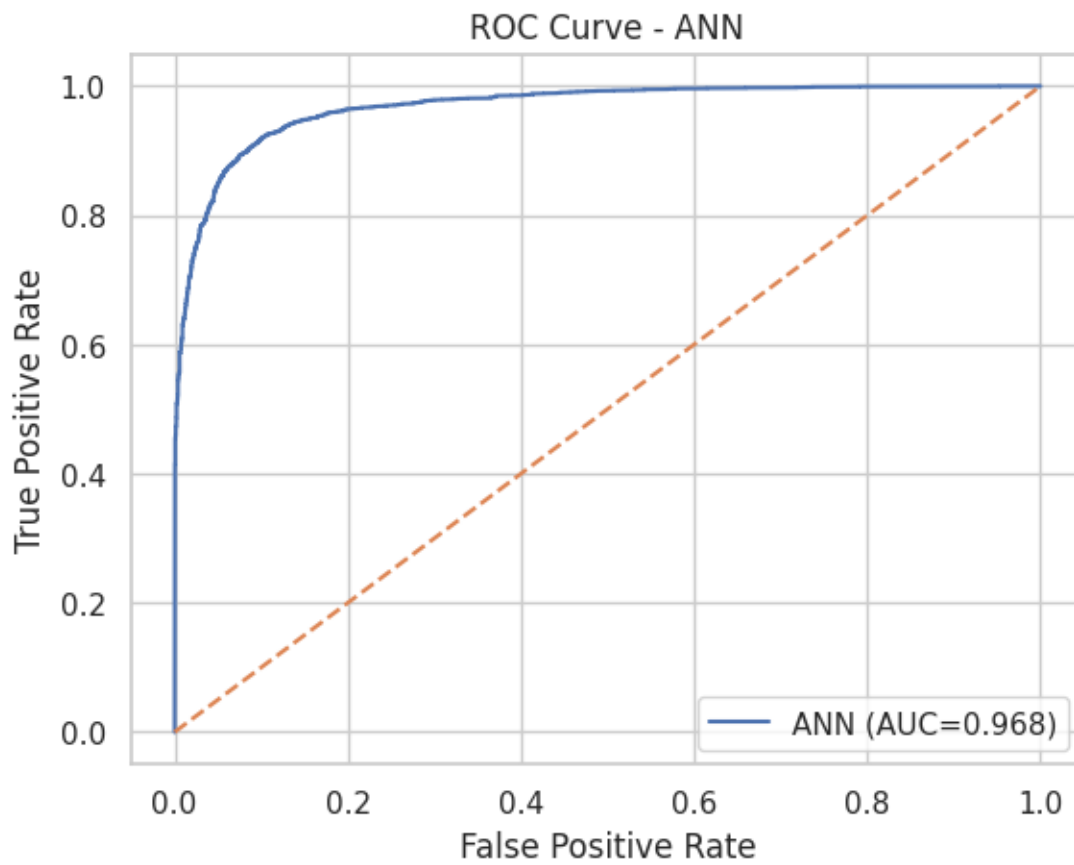
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(f"ROC Curve - {name}")
    plt.legend()

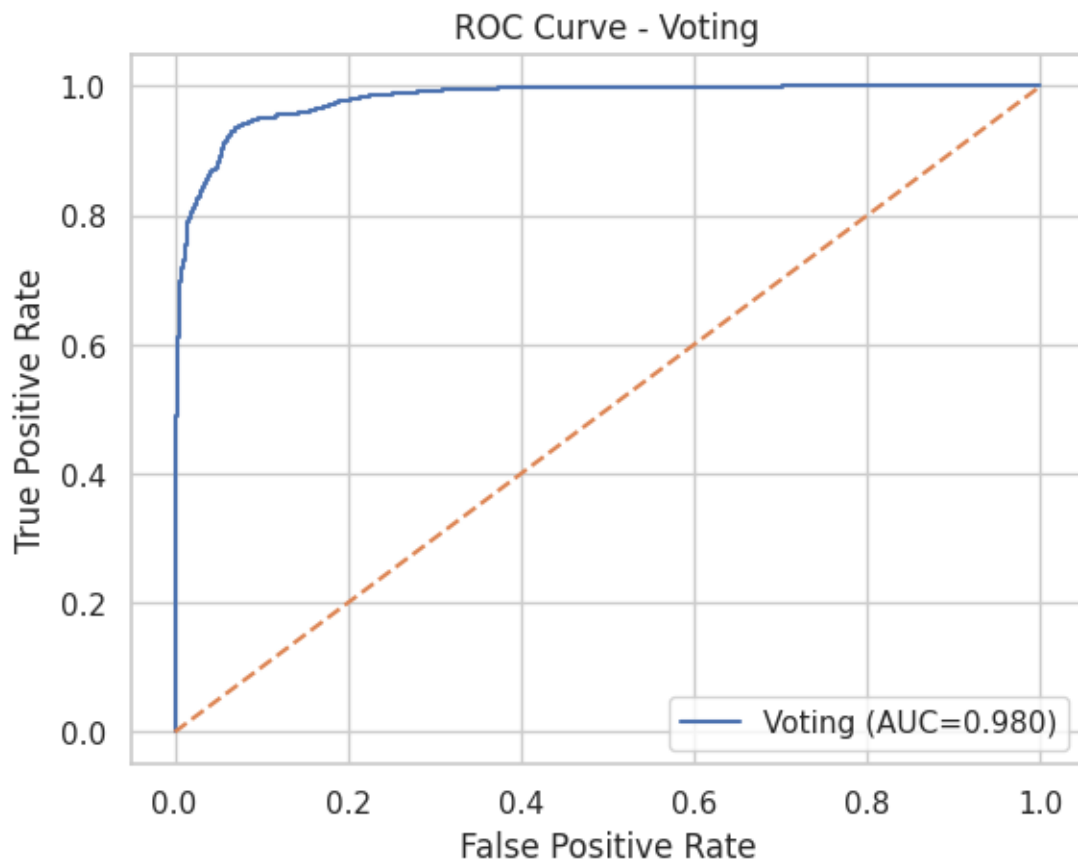
```

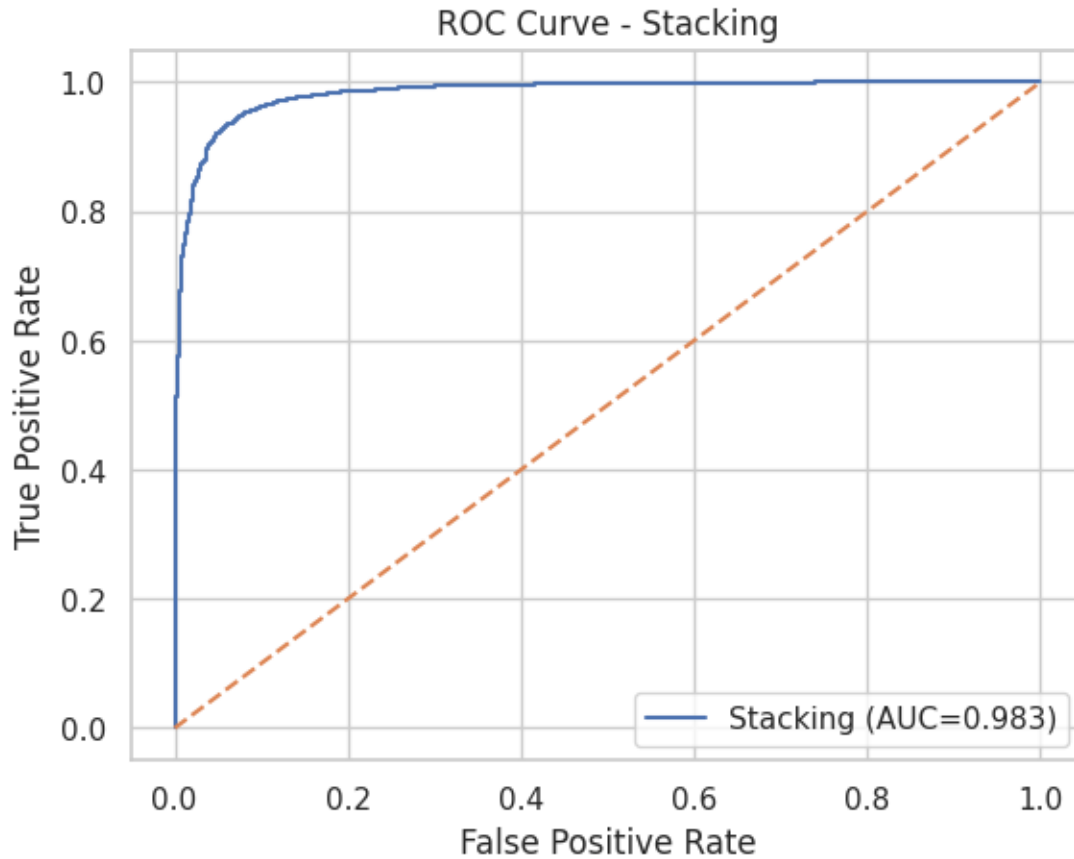
```
plt.show()
```











The ROC curves show that all models perform better than random classification, as their curves lie above the diagonal baseline. The Random Forest achieves the highest AUC of 0.98, followed by ANN at 0.97, while the Decision Tree has the lowest at 0.90.

The ANN also performs strongly by capturing nonlinear relationships but slightly underperforms compared to Random Forest. The ROC curve for the Voting model shows strong performance with an AUC of 0.980, indicating a high ability to distinguish between purchase and non-purchase sessions. The Stacking model performs slightly better, achieving the highest AUC of 0.983, with its curve closest to the top-left corner. This indicates superior classification performance and better trade-off between true positives and false positives. The improvement in Stacking is due to combining multiple models using a meta-learner, which captures more complex patterns. Overall, the results confirm that ensemble techniques enhance predictive performance, with Stacking being the best model.

```
[ ]: # COMPARISON TABLE

results = pd.DataFrame({
    'Model': ['Decision Tree', 'Random Forest', 'ANN', 'Voting', 'Stacking'],
    'Accuracy': [acc_dt, acc_rf, acc_ann, acc_vote, acc_stack],
    'Precision': [prec_dt, prec_rf, prec_ann, prec_vote, prec_stack],
```

```

    'Recall': [rec_dt, rec_rf, rec_ann, rec_vote, rec_stack],
    'F1 Score': [f1_dt, f1_rf, f1_ann, f1_vote, f1_stack],
    'AUC': [auc_dt, auc_rf, auc_ann, auc_vote, auc_stack]
})

print(results)

```

	Model	Accuracy	Precision	Recall	F1 Score	AUC
0	Decision Tree	0.897825	0.890841	0.907058	0.898876	0.897814
1	Random Forest	0.932363	0.921020	0.946024	0.933354	0.981627
2	ANN	0.908698	0.904807	0.913766	0.909264	0.968168
3	Voting	0.929165	0.917391	0.943469	0.930247	0.979717
4	Stacking	0.935881	0.930057	0.942830	0.936400	0.983230

```

[ ]: # BEST MODEL SELECTION

best_model = results.loc[results['AUC'].idxmax()]
print("\nBest Model Based on AUC:")
print(best_model)

```

Best Model Based on AUC:

```

Model          Stacking
Accuracy       0.935881
Precision      0.930057
Recall         0.94283
F1 Score       0.9364
AUC            0.98323
Name: 4, dtype: object

```

14.0.2 Model Evaluation and Comparison

The Stacking model achieved the highest performance with an accuracy of 0.936 and AUC of 0.983, outperforming all other models. Random Forest follows closely with strong performance (accuracy = 0.934, AUC = 0.982), while ANN and Voting models perform slightly lower. The Decision Tree shows the weakest performance, with lower accuracy and AUC values. The superior performance of the Stacking model is due to its ability to combine multiple learning algorithms effectively. Therefore, the Stacking model is selected as the best model for predicting purchase behavior.

15 Interaction of Variables

15.0.1 Feature Interaction Analysis

The interaction between ProductRelated_Duration and PageValues shows a clear joint influence on purchase behavior. Sessions with both high duration (e.g., above 1000 seconds) and high PageValues are significantly more likely to result in purchases. In contrast, sessions with low values in both variables rarely lead to conversion. This indicates that engagement and perceived value together

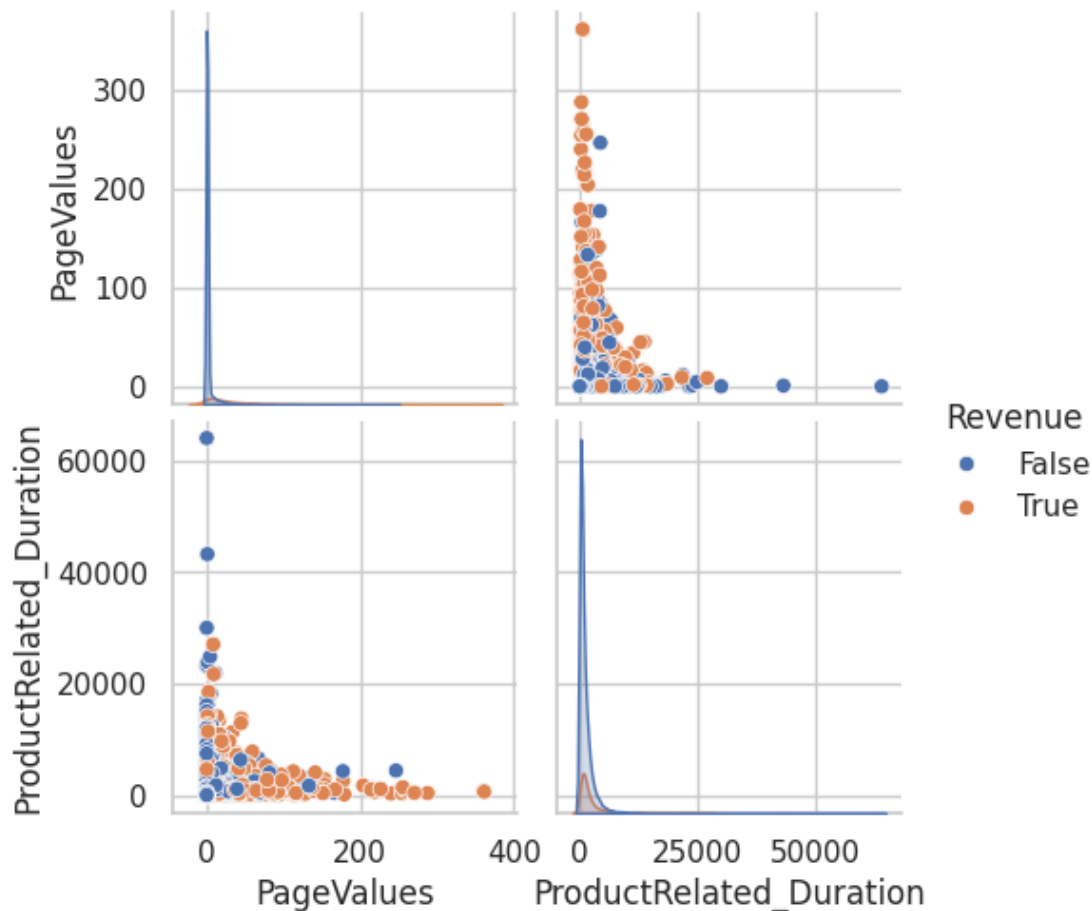
drive purchase decisions. Such nonlinear interactions explain why models like Random Forest outperform simpler models.

16 Nonlinear Relationships (Pairplots)

16.0.1 Nonlinear Relationships

Pairplots reveal that the relationship between key variables and Revenue is not strictly linear. Clusters of purchase cases appear at higher values of PageValues and ProductRelated_Duration. The overlap between classes indicates that no single feature determines purchase behavior. This supports the use of nonlinear models such as ANN and Random Forest. These models better capture complex interactions within the dataset.

```
[5]: sns.pairplot(df[['PageValues', 'ProductRelated_Duration', 'Revenue']],  
             hue='Revenue')  
plt.show()
```



The pairplot shows a clear nonlinear relationship between PageValues, ProductRelated_Duration, and Revenue. Purchase cases (Revenue = 1) are concentrated at higher PageValues, while non-

purchase cases cluster near zero. The relationship is not linear, as similar durations can result in different outcomes depending on PageValues. The overlap between classes indicates that no single variable determines purchase behavior. This justifies the use of nonlinear models such as Random Forest and ANN.

17 Conclusion

The study developed and evaluated multiple machine learning models to predict online purchase behavior using user session data. Among all models, the Stacking model achieved the highest overall performance, with the best accuracy (0.936) and AUC (0.983), indicating superior ability to distinguish between purchase and non-purchase sessions. Random Forest also performed strongly, achieving high accuracy and AUC, while ANN captured nonlinear relationships effectively. The Decision Tree provided interpretability but showed weaker generalization. Feature analysis revealed that PageValues and ProductRelated_Duration are the most influential predictors, highlighting the importance of user engagement and perceived content value in driving conversions. Hyperparameter tuning improved model robustness, and ensemble techniques such as Voting and Stacking enhanced predictive performance. Overall, the results confirm that combining models leads to better generalization and more accurate predictions in e-commerce settings.

18 Limitations and Future Work

The dataset does not include temporal or user-specific behavioral history, which may limit prediction accuracy. SMOTE introduces synthetic data, which may not fully reflect real-world patterns. The ANN model was not extensively tuned, which may affect its performance. Future work could include hyperparameter tuning, advanced models (e.g., XGBoost), and feature engineering. Incorporating time-series or user-level data could further improve predictive performance.

19 References

- Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. Online Shoppers Purchasing Intention Dataset. Retrieved from <https://archive.ics.uci.edu/ml/datasets/Online+Shoppers+Purchasing+Intention+Dataset>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Austin Peay State University. (2026). STAT 5290/4290: Consumer Behavior Modeling in E-Commerce – Course Materials.
- Masum, M. (2026). *Lecture Notes on Machine Learning Models, PCA, and Clustering*. Austin Peay State University.

[]:

[]: